```python
import tkinter as tk
from tkinter import messagebox
import database
import logic

class KioskGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Salone Kiosk Ledger - CSV Edition")
        self.root.geometry("600x650")
        self.root.configure(bg="#f4f6f9")

        # Initialize the CSV file on startup
        database.init_db()

        self.build_ui()

    def build_ui(self):
        # Header
        tk.Label(self.root, text="SALONE KIOSK LEDGER", font=("Tahoma", 16, "bold"), bg="#0072CE", fg="white", pady=10).pack(fill=tk.X)

        # Input Frame
        frame_input = tk.Frame(self.root, bg="#f4f6f9", pady=15)
        frame_input.pack()

        tk.Label(frame_input, text="Customer / Item:", font=("Tahoma", 10), bg="#f4f6f9").grid(row=0, column=0, sticky="w", padx=5, pady=5)
        self.entry_name = tk.Entry(frame_input, width=30, font=("Tahoma", 10))
        self.entry_name.grid(row=0, column=1, padx=5, pady=5)

        tk.Label(frame_input, text="Amount (SLE):", font=("Tahoma", 10), bg="#f4f6f9").grid(row=1, column=0, sticky="w", padx=5, pady=5)
        self.entry_amt = tk.Entry(frame_input, width=
```

```
30, font=("Tahoma", 10))
        self.entry_amt.grid(row=1, column=1, padx=5, pady=5)

        tk.Label(frame_input, text="Transaction Type:", font=("Tahoma", 10), bg="#f4f6f9").grid(row=2, column=0, sticky="w", padx=5, pady=5)

        self.type_var = tk.StringVar(value="Sale")
        tk.Radiobutton(frame_input, text="Sale (Income)", variable=self.type_var, value="Sale", bg="#f4f6f9").grid(row=2, column=1, sticky="w")
        tk.Radiobutton(frame_input, text="Expense (Cost)", variable=self.type_var, value="Expense", bg="#f4f6f9").grid(row=2, column=1, sticky="e")

        # Buttons Frame
        frame_btns = tk.Frame(self.root, bg="#f4f6f9")
        frame_btns.pack(pady=10)

        tk.Button(frame_btns, text="Save Data", command=self.save_transaction, bg="#1D70B8", fg="white", font=("Tahoma", 9, "bold"), width=12).grid(row=0, column=0, padx=5)
        tk.Button(frame_btns, text="View Customers", command=self.view_data, bg="#8E44AD", fg="white", font=("Tahoma", 9, "bold"), width=15).grid(row=0, column=1, padx=5)
        tk.Button(frame_btns, text="Run Report", command=self.generate_report, bg="#1E8449", fg="white", font=("Tahoma", 9, "bold"), width=12).grid(row=0, column=2, padx=5)
        tk.Button(frame_btns, text="Exit", command=self.root.quit, bg="#C0392B", fg="white", font=("Tahoma", 9, "bold"), width=8).grid(row=0, column=3, padx=5)

        # Output Display
        tk.Label(self.root, text="System Output:", font=("Tahoma", 10, "bold"), bg="#f4f6f9").pack(
```

```
anchor="w", padx=25, pady=5)
        self.display_screen = tk.Text(self.root, height=15, width=65, font=("Courier", 10), bg="#1e1e1e", fg="#00ff00")
        self.display_screen.pack(padx=20, pady=5)
        self.print_to_screen("System Ready. Database connected.\n")

    def save_transaction(self):
        name = self.entry_name.get()
        amount_raw = self.entry_amt.get()
        trans_type = self.type_var.get()

        # Validate using logic.py
        is_valid, result = logic.validate_input(name, amount_raw)

        if not is_valid:
            messagebox.showerror("Input Error", result)
            return

        # Save using database.py
        database.add_transaction(name, result, trans_type)

        self.print_to_screen(f"Saved: {name} -> SLE {result:.2f} ({trans_type})\n", clear=False)
        self.entry_name.delete(0, tk.END)
        self.entry_amt.delete(0, tk.END)

    def view_data(self):
        """Fetches and displays all data from the CSV file."""
        records = database.get_all_transactions()

        if not records:
            self.print_to_screen("The database is currently empty.\n", clear=True)
            return

```

```
82          output =
    "================================================\n
    "
83          output += f"{'CUSTOMER / ITEM'.ljust(20)} |
    {'TYPE'.ljust(10)} | {'AMOUNT'}\n"
84          output +=
    "================================================\n
    "
85
86          for r in records:
87              output += f"{r['Customer/Item'].ljust(20
    )} | {r['Type'].ljust(10)} | SLE {r['Amount']:.2f}\n
    "
88
89          output +=
    "================================================\n
    "
90          self.print_to_screen(output, clear=True)
91
92      def generate_report(self):
93          records = database.get_all_transactions()
94          if not records:
95              messagebox.showinfo("Empty", "No data to
     calculate.")
96              return
97
98          sales, expenses, gross, tax, net = logic.
    calculate_totals(records)
99          health = logic.evaluate_health(net)
100
101         report =  "--- FINANCIAL AUDIT REPORT ---\n"
102         report += f"Total Sales   : SLE {sales:.2f}\
    n"
103         report += f"Total Expenses: SLE {expenses:.
    2f}\n"
104         report += "------------------------------\n"
105         report += f"Gross Profit  : SLE {gross:.2f}\
    n"
106         report += f"Est. Tax (5%) : SLE {tax:.2f}\n"
107         report += "==============================\n"
108         report += f"NET PROFIT    : SLE {net:.2f}\n"
```

```
109         report += f"STATUS         : {health}\n"
110 
111         self.print_to_screen(report, clear=True)
112 
113     def print_to_screen(self, text, clear=False):
114         self.display_screen.config(state=tk.NORMAL)
115         if clear:
116             self.display_screen.delete(1.0, tk.END)
117         self.display_screen.insert(tk.END, text)
118         self.display_screen.config(state=tk.DISABLED
    )
119 
120 if __name__ == "__main__":
121     app_window = tk.Tk()
122     app = KioskGUI(app_window)
123     app_window.mainloop()
```